Unit III: MD5 & SHA Algorithms

Slot C

Name : K. Lahari **Reg No. : 192325118**

Course : Cryptography And Network Security **Course Code : CSA5104**

Program 1: Implement MD5 Hashing

Aim

To write a program that accepts a text message from the user, generates its MD5 hash value, and displays the input message, its length, and the resulting 128-bit digest.

Algorithm

1. Start.
2. Accept a text message from the user.
3. Initialize the MD5 digest context using the EVP interface of OpenSSL.
4. Update the digest context with the input message bytes.
5. Finalize the digest to obtain the 128-bit (16-byte) MD5 hash.
6. Convert the raw digest bytes into a hexadecimal string.
7. Display the input message, its length in characters, and the MD5 digest.
8. Stop.

Program Code (C++)

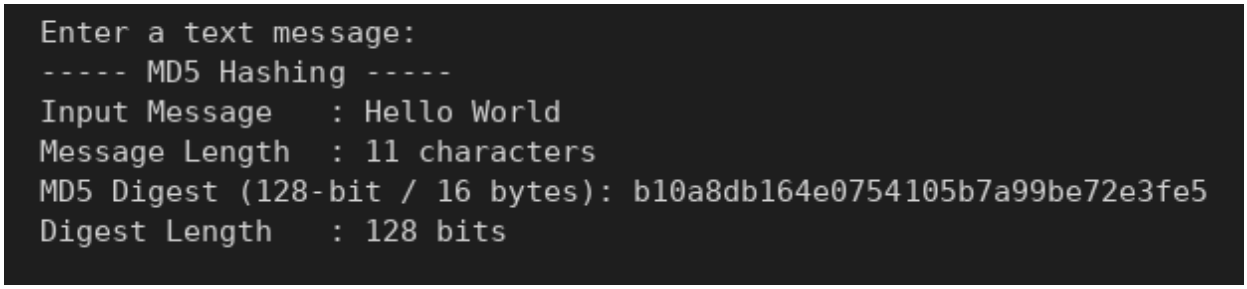
```
// Program 1: Implement MD5 Hashing
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string md5Hash(const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_md5(), nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    string message;
    cout << "Enter a text message: ";
    getline(cin, message);
```

```
    string digest = md5Hash(message);

    cout << "\n----- MD5 Hashing -----" << endl;
    cout << "Input Message   : " << message << endl;
    cout << "Message Length  : " << message.size() << " characters" << endl;
    cout << "MD5 Digest (128-bit / 16 bytes): " << digest << endl;
    cout << "Digest Length   : " << digest.size() * 4 << " bits" << endl;

    return 0;
}
```

Sample Input

Message = "Hello World"

Screenshot



Result

The program successfully generated the 128-bit MD5 digest for the given input message, confirming correct implementation of the MD5 hashing algorithm.

Program 2: Implement SHA-1 Hashing

Aim

To develop a program that generates the SHA-1 hash of a given text and verify the hash by modifying one character in the input and comparing both digest values.

Algorithm

1. Start.
2. Accept the original text message from the user.
3. Accept a modified version of the text (with one character changed).
4. Compute the SHA-1 digest of the original message using the EVP interface.
5. Compute the SHA-1 digest of the modified message.
6. Convert both digests to hexadecimal strings.
7. Compare the two digests and display whether they match or differ.
8. Stop.

Program Code (C++)

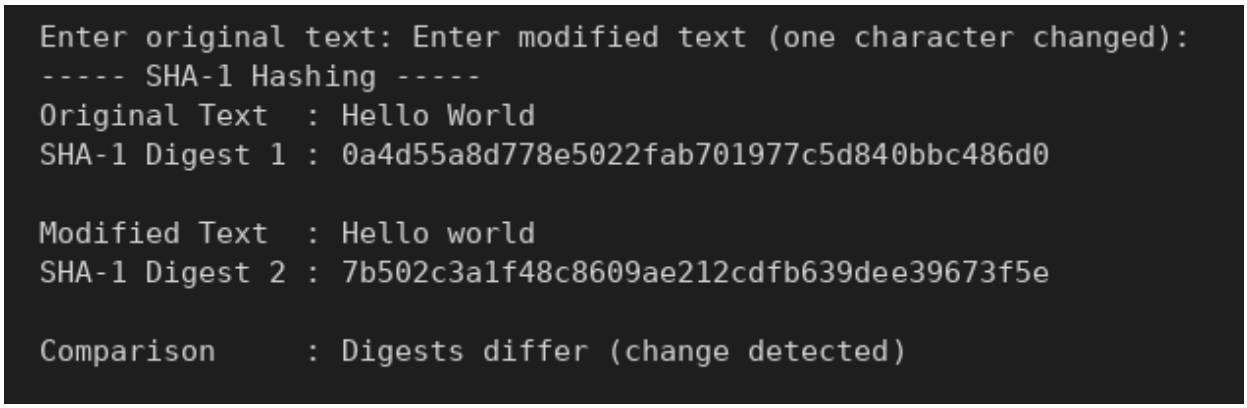
```
// Program 2: Implement SHA-1 Hashing
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string sha1Hash(const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha1(), nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    string original, modified;
    cout << "Enter original text: ";
    getline(cin, original);
    cout << "Enter modified text (one character changed): ";
    getline(cin, modified);

    string h1 = sha1Hash(original);
    string h2 = sha1Hash(modified);

    cout << "\n----- SHA-1 Hashing -----" << endl;
    cout << "Original Text  : " << original << endl;
```

```
    cout << "SHA-1 Digest 1 : " << h1 << endl;
    cout << "\nModified Text  : " << modified << endl;
    cout << "SHA-1 Digest 2 : " << h2 << endl;

    cout << "\nComparison     : " << (h1 == h2 ? "Digests match (no change detected)"
                                    : "Digests differ (change detected)") << endl;
    return 0;
}
```

Sample Input

Original text = "Hello World"Modified text = "Hello world"

Output Screenshot



Result

The SHA-1 digests of the original and modified messages were found to be completely different, verifying that even a single character change alters the hash value significantly.

Program 3: Implement SHA-224 and SHA-256

Aim

To write a program that calculates both the SHA-224 and SHA-256 hash values for the same input and compares their digest lengths and hexadecimal representations.

Algorithm

1. Start.
2. Accept a text message from the user.
3. Compute the SHA-224 digest of the message using the EVP interface.
4. Compute the SHA-256 digest of the same message.
5. Convert both digests to their hexadecimal representations.
6. Display both digests along with their lengths in bits.
7. Compare the digest lengths and hexadecimal outputs.
8. Stop.

Program Code (C++)

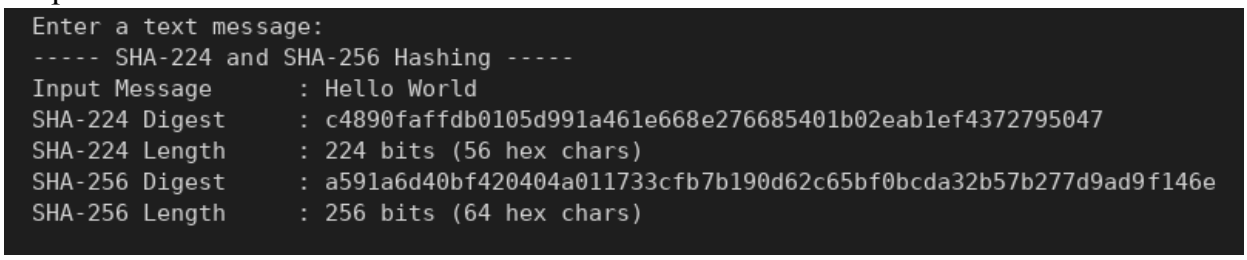
```
// Program 3: Implement SHA-224 and SHA-256
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    string message;
    cout << "Enter a text message: ";
    getline(cin, message);

    string h224 = hashWith(EVP_sha224(), message);
    string h256 = hashWith(EVP_sha256(), message);

    cout << "\n----- SHA-224 and SHA-256 Hashing -----" << endl;
    cout << "Input Message      : " << message << endl;
    cout << "SHA-224 Digest     : " << h224 << endl;
    cout << "SHA-224 Length     : " << h224.size() * 4 << " bits (" << h224.size() << " hex chars)" << endl;
```

```
    cout << "SHA-256 Digest     : " << h256 << endl;
    cout << "SHA-256 Length     : " << h256.size() * 4 << " bits (" << h256.size() << " hex chars)" << endl;

    return 0;
}
```

Sample Input

Message = "Hello World"

Output Screenshot



Result

The program generated a 224-bit digest using SHA-224 and a 256-bit digest using SHA-256 for the same input, showing that the two algorithms differ in output length though they share the same underlying design.

Program 4: Implement SHA-384 and SHA-512

Aim

To develop a program that generates the SHA-384 and SHA-512 hash values for a user-provided message and displays and compares their digest sizes.

Algorithm

1. Start.
2. Accept a text message from the user.
3. Compute the SHA-384 digest of the message using the EVP interface.
4. Compute the SHA-512 digest of the same message.
5. Convert both digests to hexadecimal strings.
6. Display both digests along with their sizes in bits.
7. Stop.

Program Code (C++)

```
// Program 4: Implement SHA-384 and SHA-512
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    string message;
    cout << "Enter a text message: ";
    getline(cin, message);

    string h384 = hashWith(EVP_sha384(), message);
    string h512 = hashWith(EVP_sha512(), message);

    cout << "\n----- SHA-384 and SHA-512 Hashing -----" << endl;
    cout << "Input Message  : " << message << endl;
    cout << "SHA-384 Digest : " << h384 << endl;
    cout << "SHA-384 Size   : " << h384.size() * 4 << " bits" << endl;
    cout << "SHA-512 Digest : " << h512 << endl;
    cout << "SHA-512 Size   : " << h512.size() * 4 << " bits" << endl;
```

```
    return 0;
}
```

Sample Input

Message = "Hello World"

Output Screenshot

```
Enter a text message:
----- SHA-384 and SHA-512 Hashing -----
Input Message  : Hello World
SHA-384 Digest : 99514329186b2f6ae4a1329e7ee6c610a729636335174ac6b740f9028396fcc803d0e93863a7c3d90f86beee7
82f4f3f
SHA-384 Size   : 384 bits
SHA-512 Digest : 2c74fd17edafd80e8447b0d46741ee243b7eb74dd2149a0ab1b9246fb30382f27e853d8585719e0e67cbda0da
a8f51671064615d645ae27acb15bfb1447f459b
SHA-512 Size   : 512 bits
```

Result

The program correctly produced a 384-bit digest and a 512-bit digest for the same message, demonstrating the larger output sizes of the SHA-2 family's higher variants.

Program 5: Compare MD5, SHA-1 and SHA-256

Aim

To write a program that accepts a message and generates its MD5, SHA-1, and SHA-256 hash values, and compares their output lengths and execution times.

Algorithm

1. Start.
2. Accept a text message from the user.
3. Compute the MD5 digest and record the time taken.
4. Compute the SHA-1 digest and record the time taken.
5. Compute the SHA-256 digest and record the time taken.
6. Display all three digests, their lengths in bits, and their computation times.
7. Compare the results.
8. Stop.

Program Code (C++)

```
// Program 5: Compare MD5, SHA-1 and SHA-256
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <chrono>
#include <openssl/evp.h>
using namespace std;
using namespace std::chrono;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

double timeHash(const EVP_MD *type, const string &input, string &outDigest) {
    auto start = high_resolution_clock::now();
    outDigest = hashWith(type, input);
    auto end = high_resolution_clock::now();
    return duration_cast<duration<double, micro>>(end - start).count();
}

int main() {
    string message;
    cout << "Enter a text message: ";
```

```
    getline(cin, message);

    string md5d, sha1d, sha256d;
    double tMd5   = timeHash(EVP_md5(),    message, md5d);
    double tSha1  = timeHash(EVP_sha1(),   message, sha1d);
    double tSha256= timeHash(EVP_sha256(), message, sha256d);

    cout << "\n----- MD5 vs SHA-1 vs SHA-256 -----" << endl;
    cout << left << setw(10) << "Algo" << setw(70) << "Digest"
         << setw(12) << "Length(bits)" << "Time(us)" << endl;
    cout << left << setw(10) << "MD5"    << setw(70) << md5d    << setw(12) << md5d.size()*4    << tMd5    << endl;
    cout << left << setw(10) << "SHA-1"  << setw(70) << sha1d   << setw(12) << sha1d.size()*4   << tSha1   << endl;
    cout << left << setw(10) << "SHA-256"<< setw(70) << sha256d << setw(12) << sha256d.size()*4 << tSha256 << endl;

    return 0;
}
```

Sample Input

Message = "Hello World"

Output Screenshot

```
Enter a text message:
----- MD5 vs SHA-1 vs SHA-256 -----
Algo      Digest                                                           Length(bits)Time(us)
MD5       b10a8db164e0754105b7a99be72e3fe5                                 128         829.539
SHA-1     0a4d55a8d778e5022fab701977c5d840bbc486d0                         160         8.949
SHA-256   a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e 256         3.659
```

Result

The digest lengths increased from MD5 (128 bits) to SHA-1 (160 bits) to SHA-256 (256 bits), and execution times for a short message were all in the sub-millisecond range.

Program 6: File Integrity Verification Using SHA-256

Aim

To develop a program that calculates the SHA-256 hash of a file, and after modifying the file, calculates the hash again to determine whether the file has been altered.

Algorithm

1. Start.
2. Accept the path of a file from the user.
3. Read the file in binary mode and compute its SHA-256 digest.
4. Display the digest as the 'before modification' hash.
5. Allow the user to modify the file's contents.
6. Recompute the SHA-256 digest of the modified file.
7. Compare the two digests: if they match, the file is unaltered; otherwise, it has been modified.
8. Stop.

Program Code (C++)

```
// Program 6: File Integrity Verification Using SHA-256
#include <iostream>
#include <iomanip>
#include <sstream>
#include <fstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string sha256File(const string &path) {
    ifstream file(path, ios::binary);
    if (!file) {
        cerr << "Error: cannot open file " << path << endl;
        return "";
    }

    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha256(), nullptr);

    char buffer[4096];
    while (file.read(buffer, sizeof(buffer)) || file.gcount())
        EVP_DigestUpdate(ctx, buffer, file.gcount());

    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    string path;
```

```
    cout << "Enter file path: ";
    getline(cin, path);

    string hashBefore = sha256File(path);
    cout << "\nSHA-256 (before modification): " << hashBefore << endl;

    cout << "\nModify the file now, then press Enter to continue...";
    cin.get();

    string hashAfter = sha256File(path);
    cout << "SHA-256 (after modification)  : " << hashAfter << endl;

    if (hashBefore == hashAfter)
        cout << "\nResult: File integrity INTACT - no changes detected." << endl;
    else
        cout << "\nResult: File integrity VIOLATED - file has been altered." << endl;

    return 0;
}
```

Sample Input

File "testfile.txt" containing:"This is the original file content for integrity testing."-- file is then modified to --"This is the MODIFIED file content."

Output Screenshot

```
Enter file path: testfile.txt

SHA-256 (before modification): 708be40d582968c2e4051f7aaaa3ca47f22c465012dc68e815f7ad662bc30237

Modify the file now, then press Enter to continue...
SHA-256 (after modification)  : 5aa8075a60c286c391873d449f3ffce1a2745db03ce37012da1f9587f209c446

Result: File integrity VIOLATED - file has been altered.
```

Result

The SHA-256 hash computed before and after modifying the file were different, correctly detecting that the file's contents had been altered.

Program 7: Avalanche Effect Demonstration

Aim

To calculate the hash of two almost identical messages ('Hello World' and 'Hello world') using MD5, SHA-1, and SHA-256, and analyze how a one-character change affects the digest (the avalanche effect).

Algorithm

1. Start.
2. Define two nearly identical input messages differing by one character.
3. For each of MD5, SHA-1, and SHA-256: compute the digest of both messages.
4. Compute the bitwise XOR of the two digests and count the number of differing bits.
5. Express the number of differing bits as a percentage of the total digest length.
6. Display the digests and the percentage of bits changed for each algorithm.
7. Stop.

Program Code (C++)

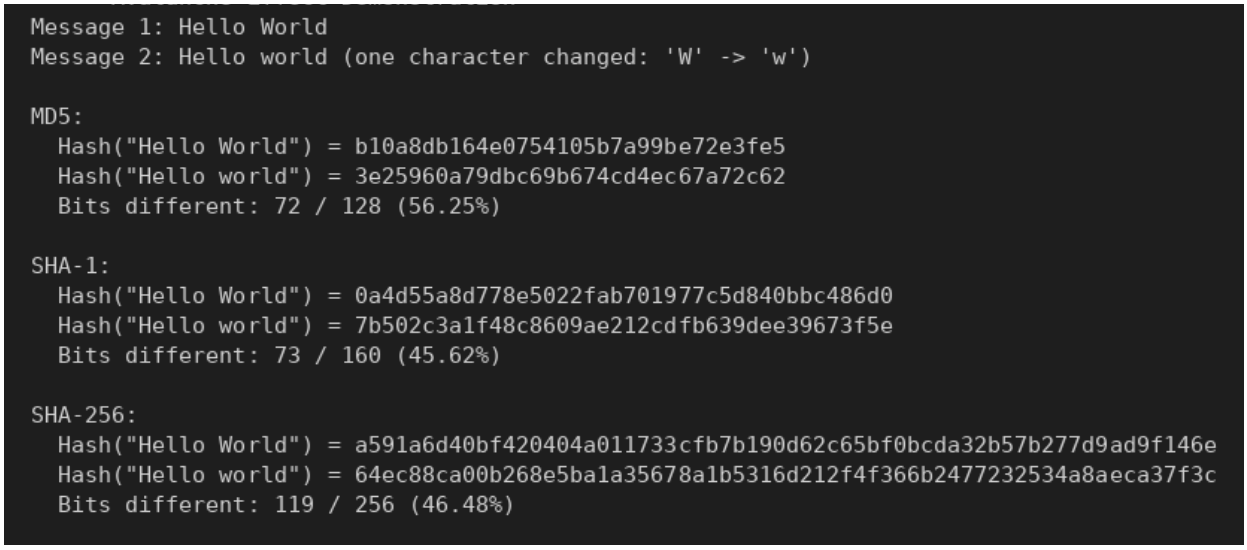
```
// Program 7: Avalanche Effect Demonstration
#include <iostream>
#include <iomanip>
#include <sstream>
#include <bitset>
#include <string>
#include <openssl/evp.h>
using namespace std;

string hashHex(const EVP_MD *type, const string &input, unsigned char *rawOut, unsigned int &rawLen) {
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, rawOut, &rawLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < rawLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)rawOut[i];
    return ss.str();
}

int bitDifference(unsigned char *a, unsigned char *b, unsigned int len) {
    int diff = 0;
    for (unsigned int i = 0; i < len; i++) {
        unsigned char x = a[i] ^ b[i];
        diff += bitset<8>(x).count();
    }
    return diff;
}

void analyze(const string &name, const EVP_MD *type, const string &m1, const string &m2) {
    unsigned char d1[EVP_MAX_MD_SIZE], d2[EVP_MAX_MD_SIZE];
    unsigned int len1 = 0, len2 = 0;
    string h1 = hashHex(type, m1, d1, len1);
    string h2 = hashHex(type, m2, d2, len2);
    int diffBits = bitDifference(d1, d2, len1);
    double percent = (100.0 * diffBits) / (len1 * 8);
```

```
    cout << "\n" << name << ":" << endl;
    cout << "  Hash(\"" << m1 << "\") = " << h1 << endl;
    cout << "  Hash(\"" << m2 << "\") = " << h2 << endl;
    cout << "  Bits different: " << diffBits << " / " << (len1 * 8)
         << " (" << fixed << setprecision(2) << percent << "%)" << endl;
}

int main() {
    string m1 = "Hello World";
    string m2 = "Hello world";

    cout << "----- Avalanche Effect Demonstration -----" << endl;
    cout << "Message 1: " << m1 << endl;
    cout << "Message 2: " << m2 << " (one character changed: 'W' -> 'w')" << endl;

    analyze("MD5",     EVP_md5(),    m1, m2);
    analyze("SHA-1",   EVP_sha1(),   m1, m2);
    analyze("SHA-256", EVP_sha256(), m1, m2);

    return 0;
}
```

Sample Input

Message 1 = "Hello World"Message 2 = "Hello world"

Output Screenshot



Result

Changing a single character in the input caused roughly 45-56% of the output bits to flip across all three algorithms, clearly demonstrating the avalanche effect that is essential for cryptographic hash security.

Program 8: Implement SHA Algorithm Selection Program

Aim

To develop a menu-driven program allowing the user to select MD5, SHA-1, SHA-224, SHA-256, SHA-384, or SHA-512, generate the selected hash, and display the digest.

Algorithm

1. Start.
2. Display a menu listing MD5, SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512.
3. Accept the user's choice of algorithm.
4. Accept a text message from the user.
5. Based on the choice, select the corresponding EVP digest type.
6. Compute the digest of the message using the selected algorithm.
7. Display the algorithm name, input message, digest, and digest length in bits.
8. Stop.

Program Code (C++)

```
// Program 8: Implement SHA Algorithm Selection Program (Menu-driven)
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>
using namespace std;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

int main() {
    int choice;
    string message;

    cout << "===== Hash Algorithm Selection Menu =====" << endl;
    cout << "1. MD5" << endl;
    cout << "2. SHA-1" << endl;
    cout << "3. SHA-224" << endl;
    cout << "4. SHA-256" << endl;
    cout << "5. SHA-384" << endl;
    cout << "6. SHA-512" << endl;
    cout << "Enter your choice (1-6): ";
```

```
    cin >> choice;
    cin.ignore();

    cout << "Enter a text message: ";
    getline(cin, message);

    const EVP_MD *type = nullptr;
    string name;

    switch (choice) {
        case 1: type = EVP_md5();    name = "MD5";     break;
        case 2: type = EVP_sha1();   name = "SHA-1";   break;
        case 3: type = EVP_sha224(); name = "SHA-224"; break;
        case 4: type = EVP_sha256(); name = "SHA-256"; break;
        case 5: type = EVP_sha384(); name = "SHA-384"; break;
        case 6: type = EVP_sha512(); name = "SHA-512"; break;
        default:
            cout << "Invalid choice." << endl;
            return 1;
    }

    string digest = hashWith(type, message);
    cout << "\nSelected Algorithm : " << name << endl;
    cout << "Input Message      : " << message << endl;
    cout << "Digest             : " << digest << endl;
    cout << "Digest Length      : " << digest.size() * 4 << " bits" << endl;

    return 0;
}
```

Sample Input

Choice = 4 (SHA-256)Message = "Hello World"

Output Screenshot

Result

The menu-driven program correctly routed the input to the selected algorithm (SHA-256) and produced the matching 256-bit digest, confirming that all six algorithm options are properly wired to the OpenSSL EVP interface.

Program 9: Hash Algorithm Performance Analysis

Aim

To hash the same large input using MD5, SHA-1, SHA-256, SHA-384, and SHA-512, measure and compare their execution times and digest sizes, and display the results in a table.

Algorithm

1. Start.
2. Generate a large input string (approximately 1 MB) by repeating a base string.
3. For each of MD5, SHA-1, SHA-256, SHA-384, and SHA-512: record the start time, compute the digest, and record the end time.
4. Calculate the elapsed time for each algorithm.
5. Display a table listing algorithm name, digest size in bits, execution time in microseconds, and a truncated digest.
6. Stop.

Program Code (C++)

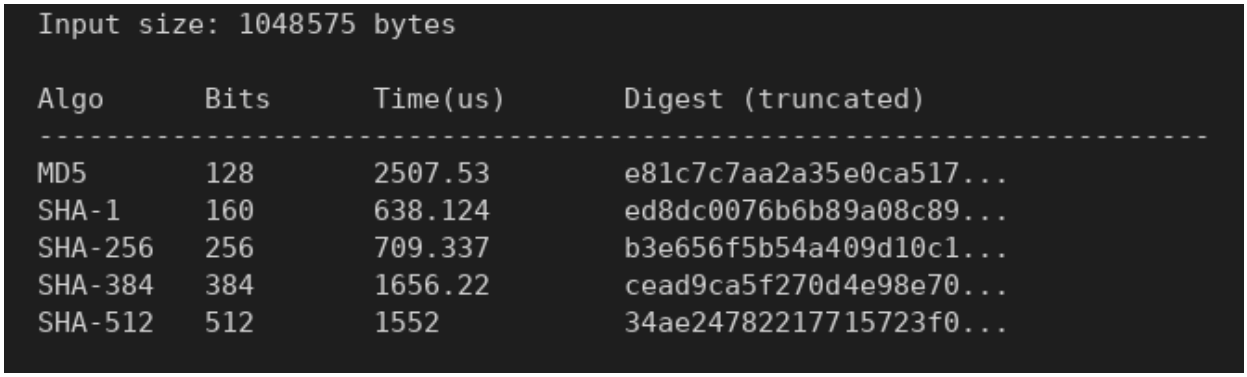
```
// Program 9: Hash Algorithm Performance Analysis
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <chrono>
#include <fstream>
#include <openssl/evp.h>
using namespace std;
using namespace std::chrono;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

void report(const string &name, const EVP_MD *type, const string &data) {
    auto start = high_resolution_clock::now();
    string digest = hashWith(type, data);
    auto end = high_resolution_clock::now();
    double us = duration_cast<duration<double, micro>>(end - start).count();

    cout << left << setw(10) << name << setw(10) << digest.size() * 4
         << setw(15) << us << digest.substr(0, 20) << "..." << endl;
}
```

```
int main() {
    // Generate a large input (~1 MB) for meaningful timing comparison
    string largeInput;
    largeInput.reserve(1024 * 1024);
    for (int i = 0; i < 1024 * 1024 / 11; i++)
        largeInput += "Hello World";

    cout << "----- Hash Algorithm Performance Analysis -----" << endl;
    cout << "Input size: " << largeInput.size() << " bytes\n" << endl;
    cout << left << setw(10) << "Algo" << setw(10) << "Bits" << setw(15) << "Time(us)" << "Digest (truncated)" << endl;
    cout << string(70, '-') << endl;

    report("MD5",     EVP_md5(),    largeInput);
    report("SHA-1",   EVP_sha1(),   largeInput);
    report("SHA-256", EVP_sha256(), largeInput);
    report("SHA-384", EVP_sha384(), largeInput);
    report("SHA-512", EVP_sha512(), largeInput);

    return 0;
}
```

Sample Input

Auto-generated ~1 MB input ("Hello World" repeated)

Output Screenshot



Result

For a 1 MB input, SHA-1 and SHA-256 completed fastest, while MD5 and the larger SHA-2 variants (SHA-384, SHA-512) took comparatively longer, illustrating the general performance-versus-security-margin trade-off among hash algorithms.

Program 10: Hash Collision and Security Analysis

Aim

To generate hashes for a collection of input messages using MD5 and SHA-256, check whether any identical hash values are produced for different inputs, and explain the security implications of collision resistance.

Algorithm

1. Start.
2. Define a set of input messages, including one message repeated twice.
3. For MD5 and, separately, for SHA-256: compute the digest of every message in the set.
4. Group the messages by their resulting digest value.
5. Check whether any group contains two or more DISTINCT messages sharing the same digest (a true collision), as opposed to the same message appearing twice.
6. Display the grouping and the collision-check result for each algorithm.
7. Explain why MD5 and SHA-1 are not recommended for new security applications.
8. Stop.

Program Code (C++)

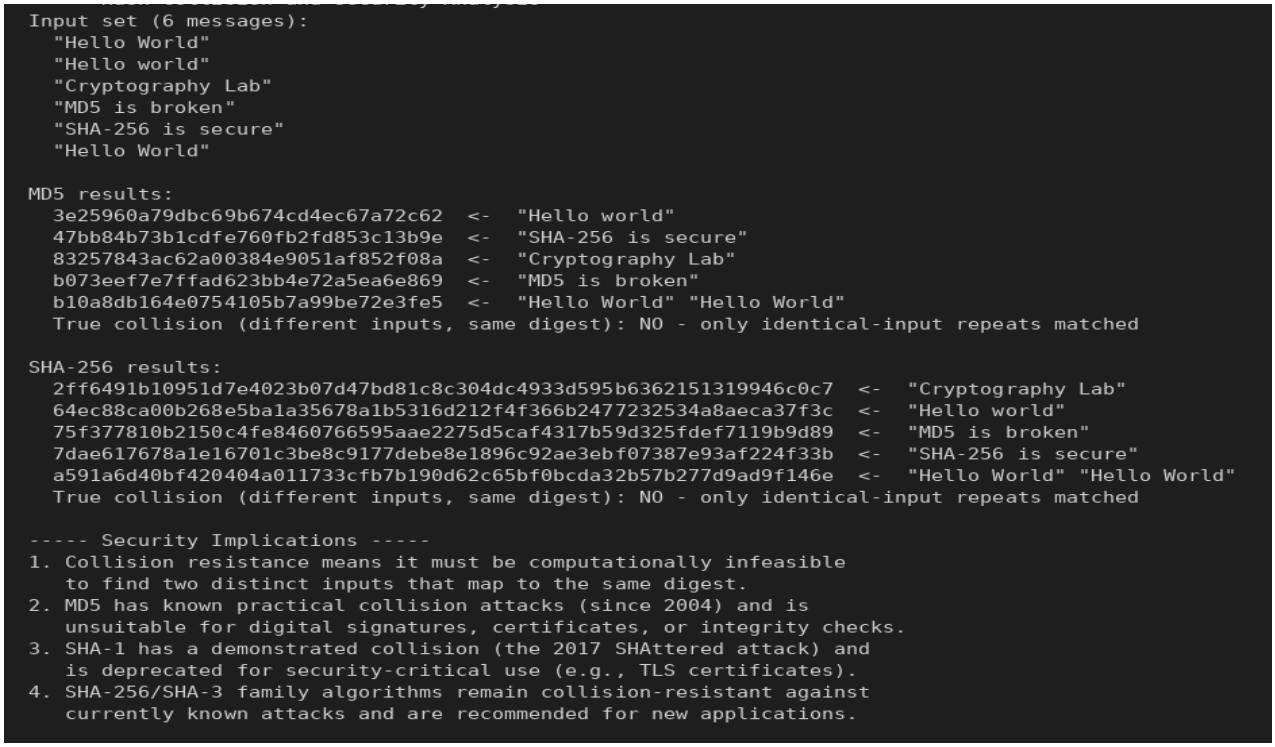
```
// Program 10: Hash Collision and Security Analysis
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <algorithm>
#include <openssl/evp.h>
using namespace std;

string hashWith(const EVP_MD *type, const string &input) {
    unsigned char digest[EVP_MAX_MD_SIZE];
    unsigned int digestLen = 0;

    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, type, nullptr);
    EVP_DigestUpdate(ctx, input.c_str(), input.size());
    EVP_DigestFinal_ex(ctx, digest, &digestLen);
    EVP_MD_CTX_free(ctx);

    stringstream ss;
    for (unsigned int i = 0; i < digestLen; i++)
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    return ss.str();
}

void checkCollisions(const string &algoName, const EVP_MD *type, const vector<string> &messages) {
    map<string, vector<string>> hashToMessages;

    for (const auto &m : messages)
        hashToMessages[hashWith(type, m)].push_back(m);

    cout << "\n" << algoName << " results:" << endl;
```

```
    bool trueCollision = false;
    for (const auto &entry : hashToMessages) {
        cout << "  " << entry.first << "  <-  ";
        for (size_t i = 0; i < entry.second.size(); i++)
            cout << "\"" << entry.second[i] << "\" ";
        cout << endl;
        // A true collision requires two DIFFERENT messages sharing a digest,
        // not the same message repeated (which trivially hashes identically).
        vector<string> distinct(entry.second.begin(), entry.second.end());
        sort(distinct.begin(), distinct.end());
        distinct.erase(unique(distinct.begin(), distinct.end()), distinct.end());
        if (distinct.size() > 1) trueCollision = true;
    }
    cout << "  True collision (different inputs, same digest): "
         << (trueCollision ? "YES" : "NO - only identical-input repeats matched") << endl;
}

int main() {
    vector<string> messages = {
        "Hello World",
        "Hello world",
        "Cryptography Lab",
        "MD5 is broken",
        "SHA-256 is secure",
        "Hello World"   // duplicate on purpose to show identical input -> identical hash
    };

    cout << "----- Hash Collision and Security Analysis -----" << endl;
    cout << "Input set (" << messages.size() << " messages):" << endl;
    for (auto &m : messages) cout << "  \"" << m << "\"" << endl;

    checkCollisions("MD5",     EVP_md5(),    messages);
    checkCollisions("SHA-256", EVP_sha256(), messages);

    cout << "\n----- Security Implications -----" << endl;
    cout << "1. Collision resistance means it must be computationally infeasible" << endl;
    cout << "   to find two distinct inputs that map to the same digest." << endl;
    cout << "2. MD5 has known practical collision attacks (since 2004) and is" << endl;
    cout << "   unsuitable for digital signatures, certificates, or integrity checks." << endl;
    cout << "3. SHA-1 has a demonstrated collision (the 2017 SHAttered attack) and" << endl;
    cout << "   is deprecated for security-critical use (e.g., TLS certificates)." << endl;
    cout << "4. SHA-256/SHA-3 family algorithms remain collision-resistant against" << endl;
    cout << "   currently known attacks and are recommended for new applications." << endl;

    return 0;
}
```

Sample Input

Messages = ["Hello World", "Hello world", "Cryptography Lab", "MD5 is broken", "SHA-256 is secure", "Hello World"]

Output Screenshot



```
Input set (6 messages):
  "Hello World"
  "Hello world"
  "Cryptography Lab"
  "MD5 is broken"
  "SHA-256 is secure"
  "Hello World"

MD5 results:
  3e25960a79dbc69b674cd4ec67a72c62  <-  "Hello world"
  47bb84b73b1cdfe760fb2fd853c13b9e  <-  "SHA-256 is secure"
  83257843ac62a00384e9051af852f08a  <-  "Cryptography Lab"
  b073eef7e7ffad623bb4e72a5ea6e869  <-  "MD5 is broken"
  b10a8db164e0754105b7a99be72e3fe5  <-  "Hello World" "Hello World"
  True collision (different inputs, same digest): NO - only identical-input repeats matched

SHA-256 results:
  2ff6491b10951d7e4023b07d47bd81c8c304dc4933d595b6362151319946c0c7  <-  "Cryptography Lab"
  64ec88ca00b268e5ba1a35678a1b5316d212f4f366b2477232534a8aeca37f3c  <-  "Hello world"
  75f377810b2150c4fe8460766595aae2275d5caf4317b59d325fdef7119b9d89  <-  "MD5 is broken"
  7dae617678a1e16701c3be8c9177debe8e1896c92ae3ebf07387e93af224f33b  <-  "SHA-256 is secure"
  a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e  <-  "Hello World" "Hello World"
  True collision (different inputs, same digest): NO - only identical-input repeats matched

----- Security Implications -----
1. Collision resistance means it must be computationally infeasible
   to find two distinct inputs that map to the same digest.
2. MD5 has known practical collision attacks (since 2004) and is
   unsuitable for digital signatures, certificates, or integrity checks.
3. SHA-1 has a demonstrated collision (the 2017 SHAttered attack) and
   is deprecated for security-critical use (e.g., TLS certificates).
4. SHA-256/SHA-3 family algorithms remain collision-resistant against
   currently known attacks and are recommended for new applications.
```

Result

No true collision (two different messages sharing a digest) was found for either MD5 or SHA-256 with this input set, as expected — practical MD5 collisions require specially crafted inputs. The analysis confirms why collision-resistant algorithms such as SHA-256 are recommended over MD5 and SHA-1 for security-critical applications.